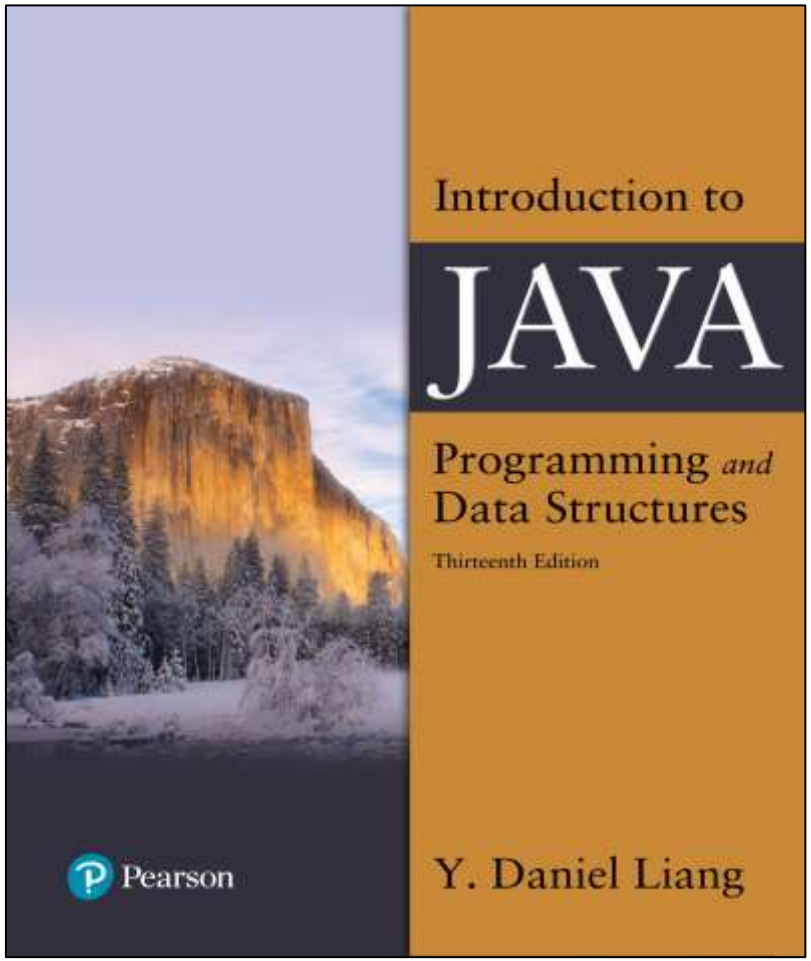
Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 10

Thinking in Objects

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Motivations

You see the advantages of object-oriented programming from the preceding chapter. This chapter will demonstrate how to solve problems using the object-oriented paradigm.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (1 of 2)

10.1 To apply class abstraction to develop software (§10.2).

10.2 To explore the differences between the procedural paradigm and object-oriented paradigm (§10.3).

10.3 To discover the relationships between classes (§10.4).

10.4 To design programs using the object-oriented paradigm (§§10.5–10.6).

10.5 To create objects for primitive values using the wrapper classes (**`Byte`**, **`Short`**, **`Integer`**, **`Long`**, **`Float`**, **`Double`**, **`Character`**, and **`Boolean`**) (§10.7).

10.6 To simplify programming using automatic conversion between primitive types and wrapper class types (§10.8).

10.7 To use the **`BigInteger`** and **`BigDecimal`** classes for computing very large numbers with arbitrary precisions (§10.9).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (2 of 2)

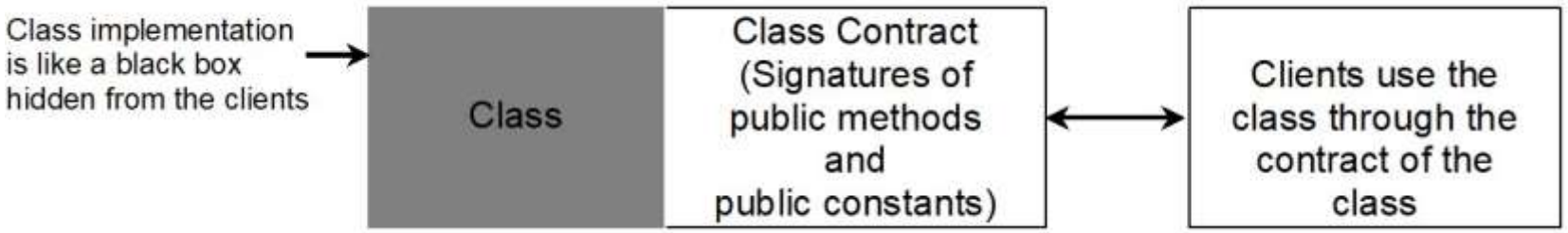
10.8 To use the **`String`** class to process immutable strings (§10.10).

10.9 To use the **`StringBuilder`** and **`StringBuffer`** classes to process mutable strings (§10.11).

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

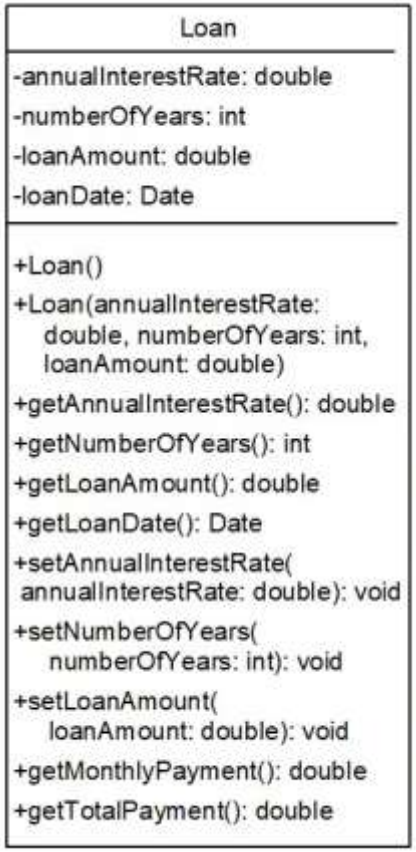
Class Abstraction and Encapsulation

Class abstraction means to separate class implementation from the use of the class. The creator of the class provides a description of the class and let the user know how the class can be used. The user of the class does not need to know how the class is implemented. The detail of implementation is encapsulated and hidden from the user.



Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Designing the `Loan` Class



The annual interest rate of the loan (default: 2.5).
The number of years for the loan (default: 1)
The loan amount (default: 1000).
The date this loan was created.

Constructs a default Loan object.
Constructs a loan with specified interest rate, years, and loan amount.
Returns the annual interest rate of this loan.
Returns the number of the years of this loan.
Returns the amount of this loan.
Returns the date of the creation of this loan.
Sets a new annual interest rate to this loan.
Sets a new number of years to this loan.
Sets a new amount to this loan.
Returns the monthly payment of this loan.
Returns the total payment of this loan.

Loan TestLoanClass

Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

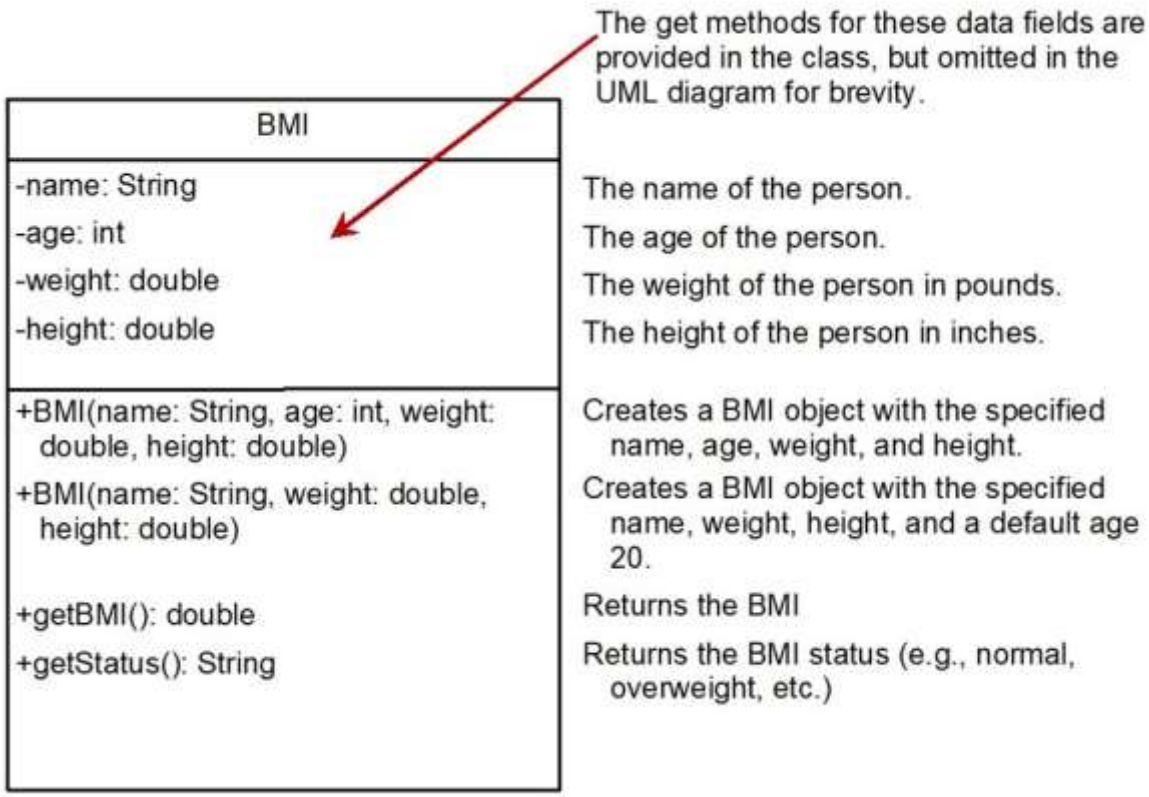
Object-Oriented Thinking

Chapters 1-8 introduced fundamental programming techniques for problem solving using loops, methods, and arrays. The studies of these techniques lay a solid foundation for object-oriented programming. Classes provide more flexibility and modularity for building reusable software. This section improves the solution for a problem introduced in Chapter 3 using the object-oriented approach. From the improvements, you will gain the insight on the differences between the procedural programming and object-oriented programming and see the benefits of developing reusable code using objects and classes.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `BMI` Class



BMI UseBMIClass

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Class Relationships

Association

Aggregation

Composition

Inheritance (Chapter 13)

Association: is a general binary relationship that describes an activity between two classes.

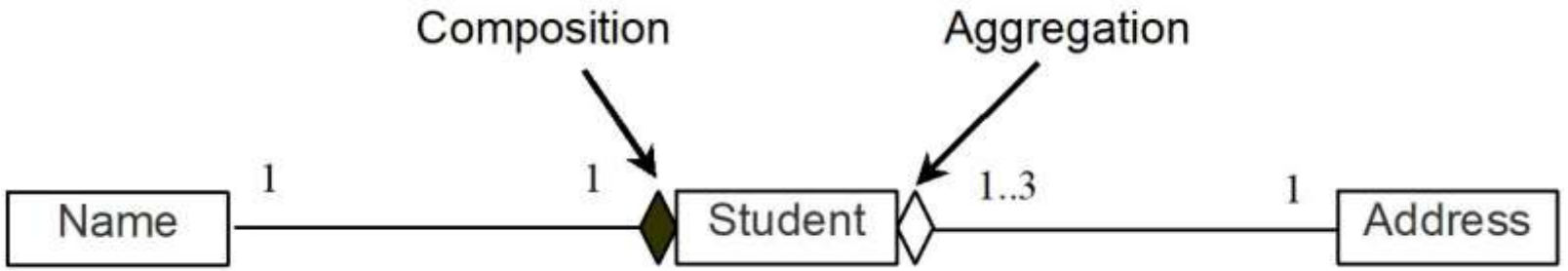


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Object Composition

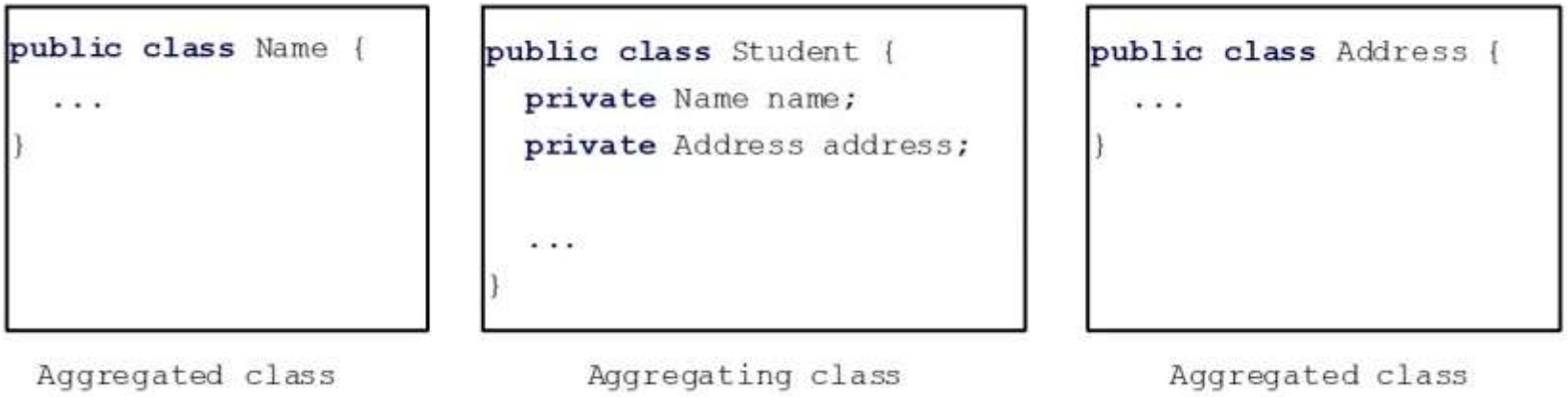
Composition is actually a special case of the aggregation relationship. Aggregation models **`has-a`** relationships and represents an ownership relationship between two objects. The owner object is called an **`aggregating object`** and its class an **`aggregating class`**. The subject object is called an **`aggregated object`** and its class an **`aggregated class`**.



Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Class Representation

An aggregation relationship is usually represented as a data field in the aggregating class. For example, the relationship in Figure 10.6 can be represented as follows:



Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Aggregation or Composition

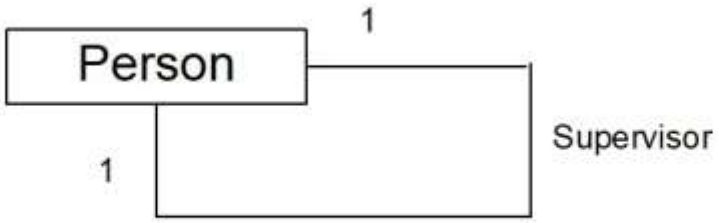
Since aggregation and composition relationships are represented using classes in similar ways, many texts don’t differentiate them and call both compositions.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Aggregation Between Same Class (1 of 2)

Aggregation may exist between objects of the same class. For example, a person may have a supervisor.



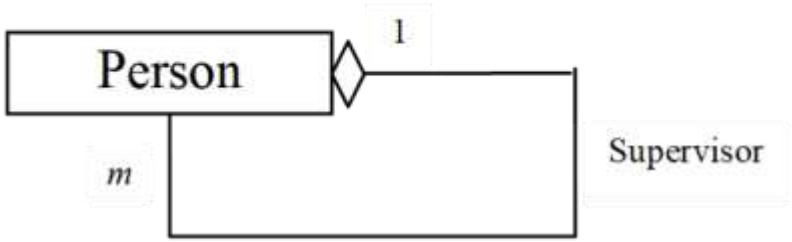

```
public class Person {
  // The type for the data is the class itself
  private Person supervisor;
  ...
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Aggregation Between Same Class (2 of 2)

What happens if a person has several supervisors?

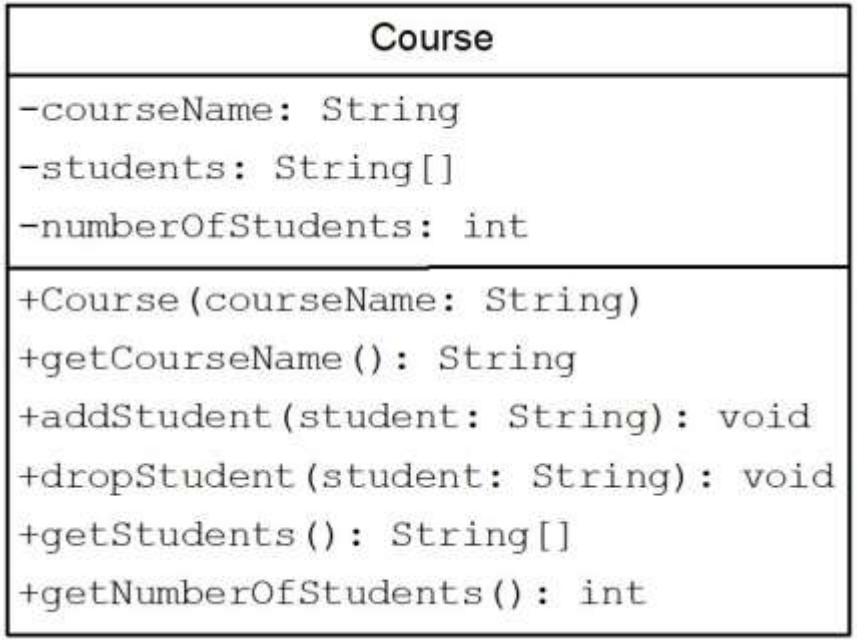



```
public class Person {
   ...
   private Person[] supervisors;
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Example: The `Course` Class



The name of the course.

An array to store the students for the course.

The number of students (default: 0).

Creates a course with the specified name.

Returns the course name.

Adds a new student to the course.

Drops a student from the course.

Returns the students in the course.

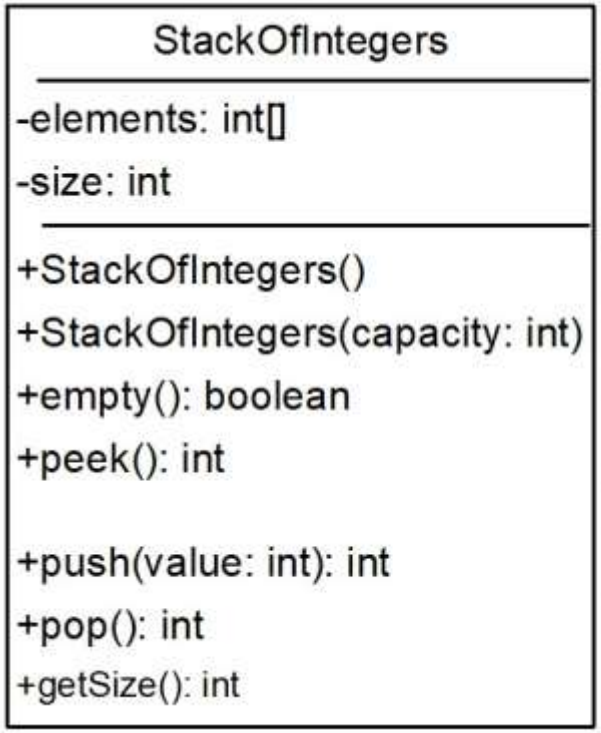
Returns the number of students in the course.

Course TestCourse

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Example: The `Stackofintegers` Class



An array to store integers in the stack.

The number of integers in the stack.

Constructs an empty stack with a default capacity of 16.

Constructs an empty stack with a specified capacity.

Returns true if the stack is empty.

Returns the integer at the top of the stack without removing it from the stack.

Stores an integer into the top of the stack.

Removes the integer at the top of the stack and returns it.

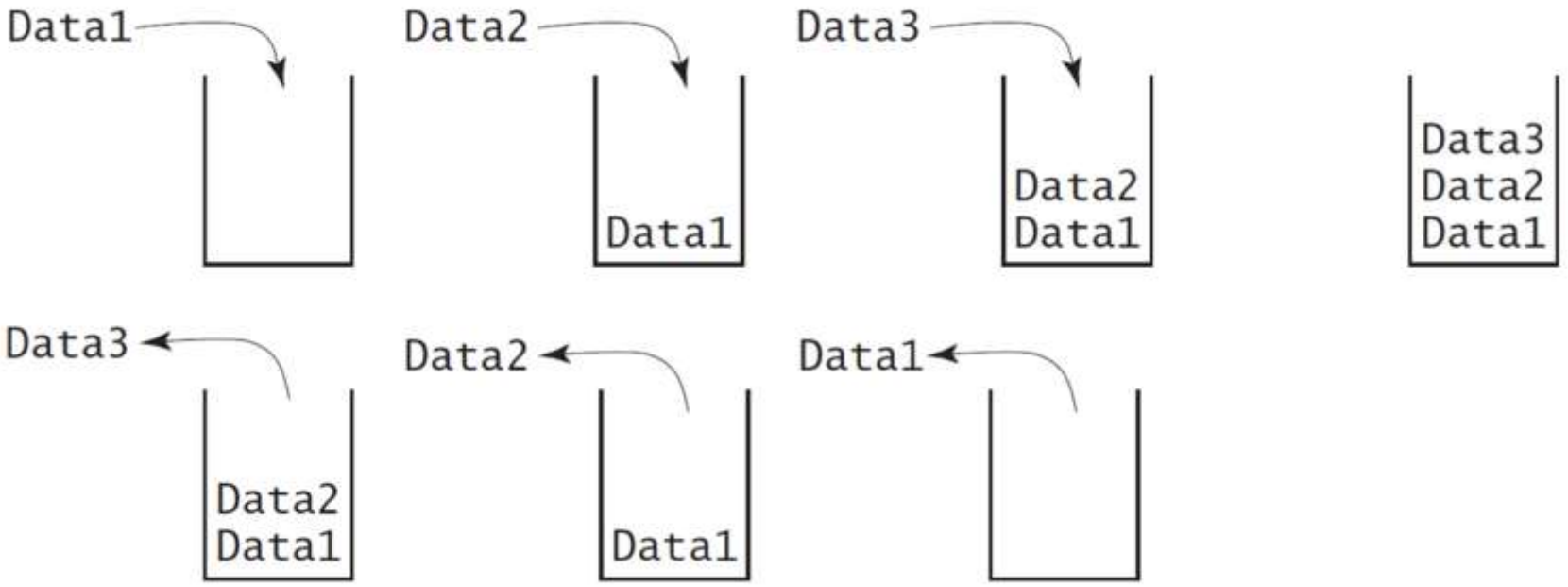
Returns the number of elements in the stack.

TestStackOfIntegers

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

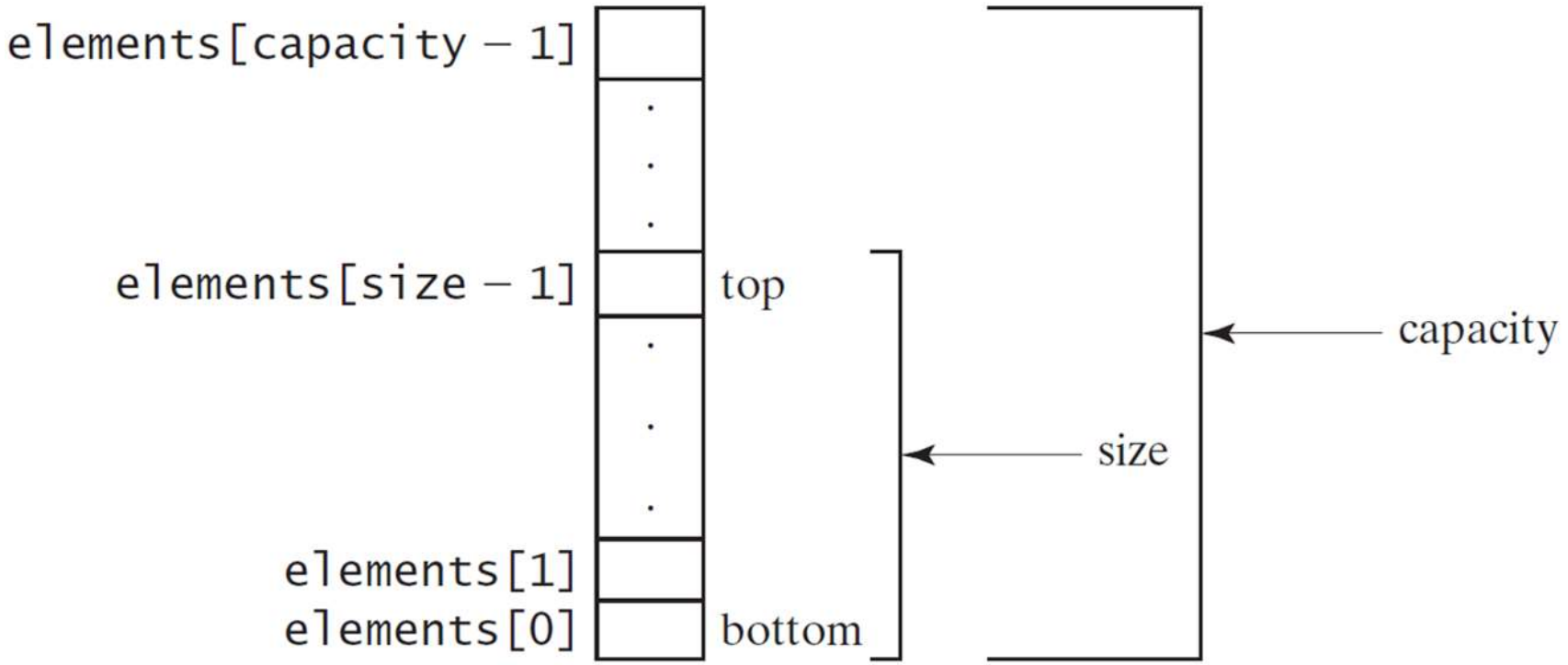
Designing the `Stackofintegers` Class



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Implementing `Stackofintegers` Class



StackOfIntegers

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Wrapper Classes

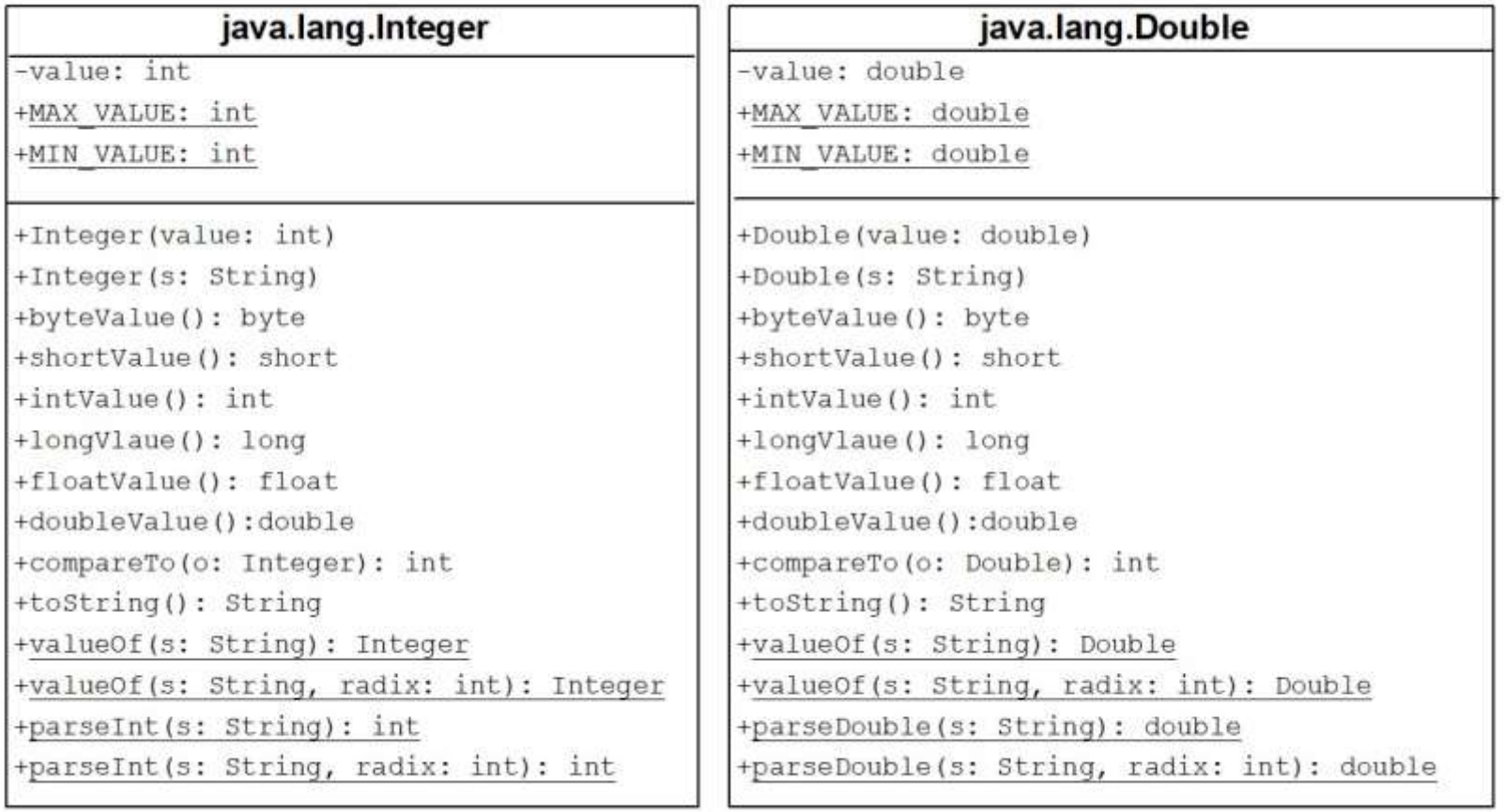
- `Boolean`
- `Character`
- `Short`
- `Byte`
- `Integer`
- `Long`
- `Float`
- `Double`

Note: (1) The wrapper classes do not have no-arg constructors. (2) The instances of all wrapper classes are immutable, i.e., their internal values cannot be changed once the objects are created.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `Integer` and `Double` Classes



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `Integer` Class and the `Double` Class

- Constructors
- Class Constants `MAX_VALUE`, `MIN_VALUE`
- Conversion Methods

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Numeric Wrapper Class Constructors

You can construct a wrapper object either from a primitive data type value or from a string representing the numeric value. The constructors for `Integer` and `Double` are:

```
public Integer(int value)
public Integer(String s)
public Double(double value)
public Double(String s)
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Numeric Wrapper Class Constants

Each numerical wrapper class has the constants **`MAX_VALUE`** and **`MIN_VALUE`**. **`MAX_VALUE`** represents the maximum value of the corresponding primitive data type. For **`Byte`**, **`Short`**, **`Integer`**, and **`Long`**, **`MIN_VALUE`** represents the minimum **`byte`**, **`short`**, **`int`**, and **`long`** values. For **`Float`** and **`Double`**, **`MIN_VALUE`** represents the minimum **positive** **`float`** and **`double`** values. The following statements display the maximum integer (2,147,483,647), the minimum positive float (1.4E-45), and the maximum double floating-point number (1.79769313486231570e+308d).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Conversion Methods

Each numeric wrapper class implements the abstract methods **`doubleValue`**, **`floatValue`**, **`intValue`**, **`longValue`**, and **`shortValue`**, which are defined in the **`Number`** class. These methods “convert” objects into primitive type values.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The Static `valueof` Methods

The numeric wrapper classes have a useful class method, `valueOf(String s)`. This method creates a new object initialized to the value represented by the specified string. For example:

```
Double doubleObject = Double.valueOf("12.4");

Integer integerObject = Integer.valueOf("12");
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The Methods for Parsing Strings Into Numbers

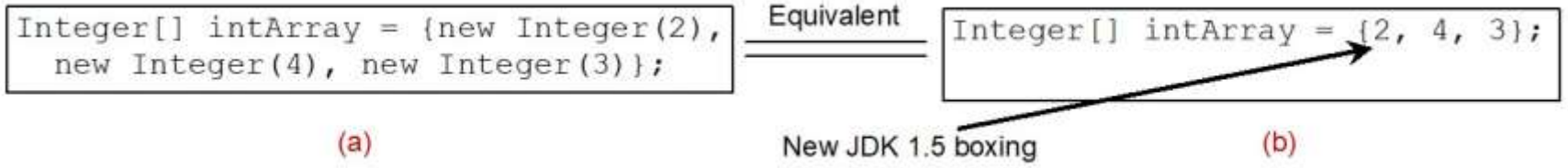
You have used the `parseInt` method in the Integer class to parse a numeric string into an int value and the `parseDouble` method in the Double class to parse a numeric string into a double value. Each numeric wrapper class has two overloaded parsing methods to parse a numeric string into an appropriate numeric value.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Automatic Conversion Between Primitive Types and Wrapper Class Types

JDK 1.5 allows primitive type and wrapper classes to be converted automatically. For example, the following statement in (a) can be simplified as in (b):




```
Integer[] intArray = {1, 2, 3};
System.out.println(intArray[0] + intArray[1] + intArray[2]);
```

Unboxing

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

`Biginteger` and `Bigdecimal` (1 of 2)

If you need to compute with very large integers or high precision floating-point values, you can use the **`BigInteger`** and **`BigDecimal`** classes in the **`java.math`** package. Both are **`immutable`**. Both extend the **`Number`** class and implement the **`Comparable`** interface.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Biginteger and Bigdecimal (2 of 2)

```
BigInteger a = new BigInteger("92233720368547758O7");

BigInteger b = new BigInteger("2");

BigInteger c = a.multiply(b); // 9223372036854775807 * 2

System.out.println(c);

BigDecimal a = new BigDecimal(1.0);

BigDecimal b = new BigDecimal(3);

BigDecimal c = a.divide(b, 20, BigDecimal.ROUND_UP);

System.out.println(c);
```

LargeFactorial

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `String` Class

- Constructing a `String`:

```
String message = "Welcome to Java";
String message = new String("Welcome to Java");
String s = new String();
```

- Obtaining String length and Retrieving Individual Characters in a string
- String Concatenation (concat)
- Substrings (`substring(index), substring(start, end)`)
- Comparisons (equals, compareTo)
- String Conversions
- Finding a `Character` or a `Substring` in a `String`
- Conversions between Strings and Arrays
- Converting Characters and Numeric Values to Strings

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Constructing Strings

```
String newString = new String(stringLiteral);

String message = new String("Welcome to Java");
```

Since strings are used frequently, Java provides a shorthand initializer for creating a string:

```
String message = "Welcome to Java";
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Strings Are Immutable

A String object is immutable; its contents cannot be changed. Does the following code change the contents of the string?

```
String s = "Java";

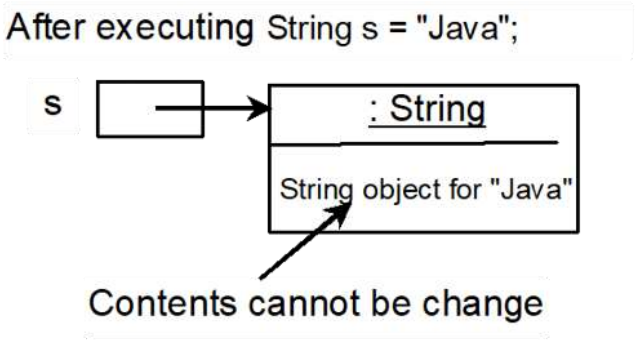
s = "HTML";
```

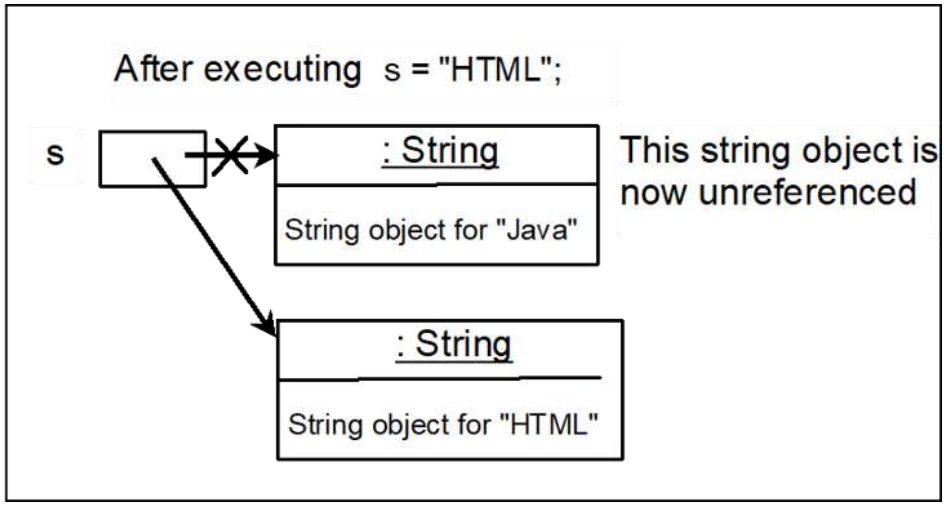
Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace Code (1 of 5)

```
String s = "Java";
s = "HTML";
```



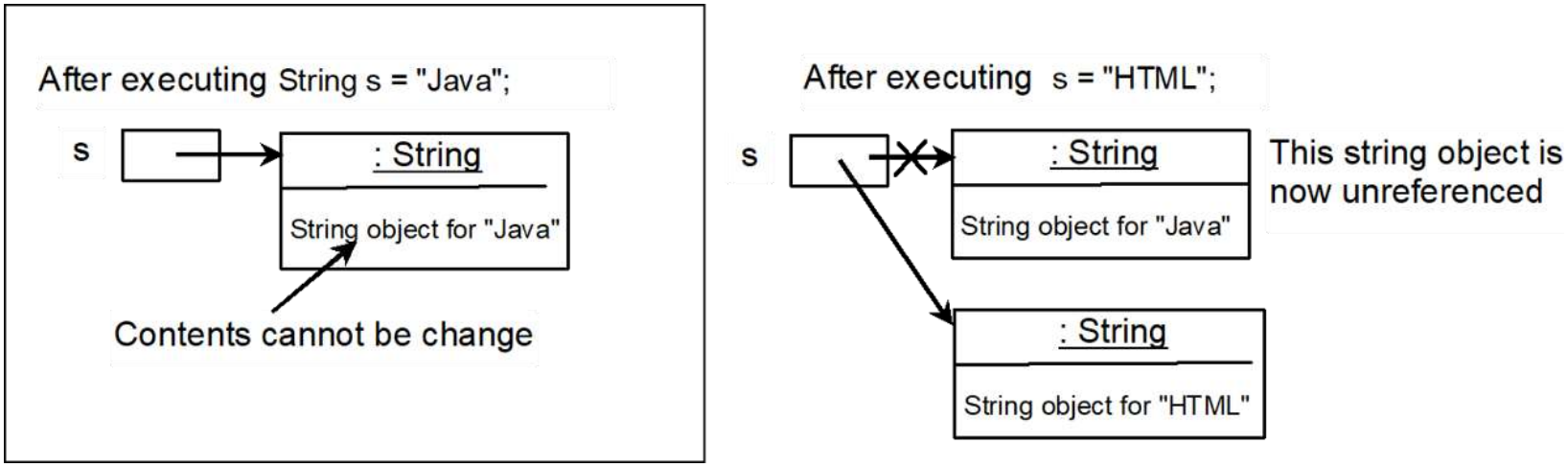


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace Code (2 of 5)

```
String s = "Java";
s = "HTML";
```



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Interned Strings

Since strings are immutable and are frequently used, to improve efficiency and save memory, the JVM uses a unique instance for string literals with the same character sequence. Such an instance is called **interned**. For example, the following statements:

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

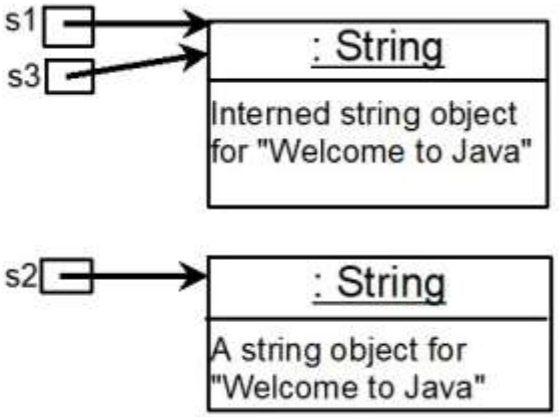
Examples (1 of 4)

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";

System.out.println("s1 == s2 is " + (s1 == s2));
System.out.println("s1 == s3 is " + (s1 == s3));
```



display

```
s1 == s is false

s1 == s3 is true
```

A new object is created if you use the new operator.

If you use the string initializer, no new object is created if the interned object is already created.

Pearson

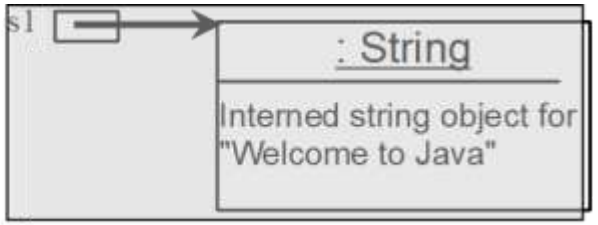
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace Code (3 of 5)

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";
```



Pearson

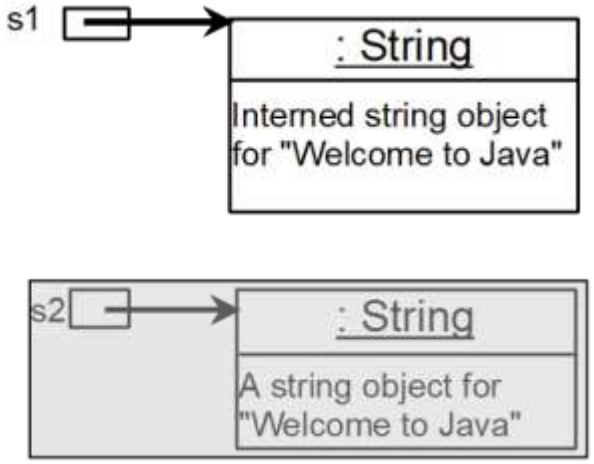
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace Code (4 of 5)

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";
```



Pearson

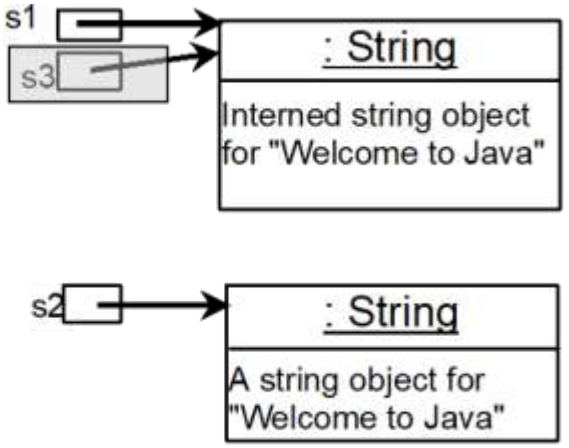
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace Code (5 of 5)

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";
```



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Replacing and Splitting Strings (1 of 2)

java.lang.String	
+replace(oldChar: char, newChar: char): String	Returns a new string that replaces all matching character in this string with the new character.
+replaceFirst(oldString: String, newString: String): String	Returns a new string that replaces the first matching substring in this string with the new substring.
+replaceAll(oldString: String, newString: String): String	Returns a new string that replace all matching substrings in this string with the new substring.
+split(delimiter: String): String[]	Returns an array of strings consisting of the substrings split by the delimiter.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Examples (2 of 4)

```
"Welcome".replace('e', 'A') returns a new string, WAlcomA.

"Welcome".replaceFirst("e", "AB") returns a new string,
WABlcome.

"Welcome".replace("e", "AB") returns a new string,
WABlcomAB.

"Welcome".replace("el", "AB") returns a new string,
WABcome.
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Splitting a String

```
String[] tokens =
"Java#HTML#Perl".split("#", 0);

for (int i = 0; i < tokens.length; i++)

  System.out.print(tokens[i] + " ");
```

displays

Java HTML Perl

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Matching, Replacing and Splitting by Patterns (1 of 3)

You can match, replace, or split a string by specifying a pattern. This is an extremely useful and powerful feature, commonly known as **regular expression**. Regular expression is complex to beginning students. For this reason, two simple patterns are used in this section. Please refer to Supplement III.F, “Regular Expressions,” for further studies.

```
"Java".matches("Java");
"Java".equals("Java");
"Java is fun".matches("Java.*");
"Java is cool".matches("Java.*");
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Matching, Replacing and Splitting by Patterns (2 of 3)

The `replaceAll`, `replaceFirst`, and split methods can be used with a regular expression. For example, the following statement returns a new string that replaces `$, +,` or `#` in `"a+b$#c"` by the string `NNN`.

```
String s = "a+b$#c".replaceAll("[$+#]", "NNN");

System.out.println(s);
```

Here the regular expression `[$+#]` specifies a pattern that matches `$, +,` or `#`. So, the output is `aNNNbNNNNNNc`.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Matching, Replacing and Splitting by Patterns (3 of 3)

The following statement splits the string into an array of strings delimited by some punctuation marks.

```
String[] tokens = "Java,C?C#,C++".split("[.,:;?]");

for (int i = 0; i < tokens.length; i++)

System.out.println(tokens[i]);
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Convert Character and Numbers to Strings

The String class provides several static `valueOf` methods for converting a character, an array of characters, and numeric values to strings. These methods have the same name `valueOf` with different argument types `char`, `char[]`, `double`, `long`, `int`, and `float`. For example, to convert a double value to a string, use `String.valueOf(5.44)`. The return value is string consists of characters `'5'`, `'.'`, `'4'`, and `'4'`.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

StringBuilder and StringBuffer

The `StringBuilder/StringBuffer` class is an alternative to the `String` class. In general, a `StringBuilder/StringBuffer` can be used wherever a `string` is used. `StringBuilder/StringBuffer` is more flexible than String. You can add, insert, or append new contents into a string buffer, whereas the value of a String object is fixed once the string is created.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

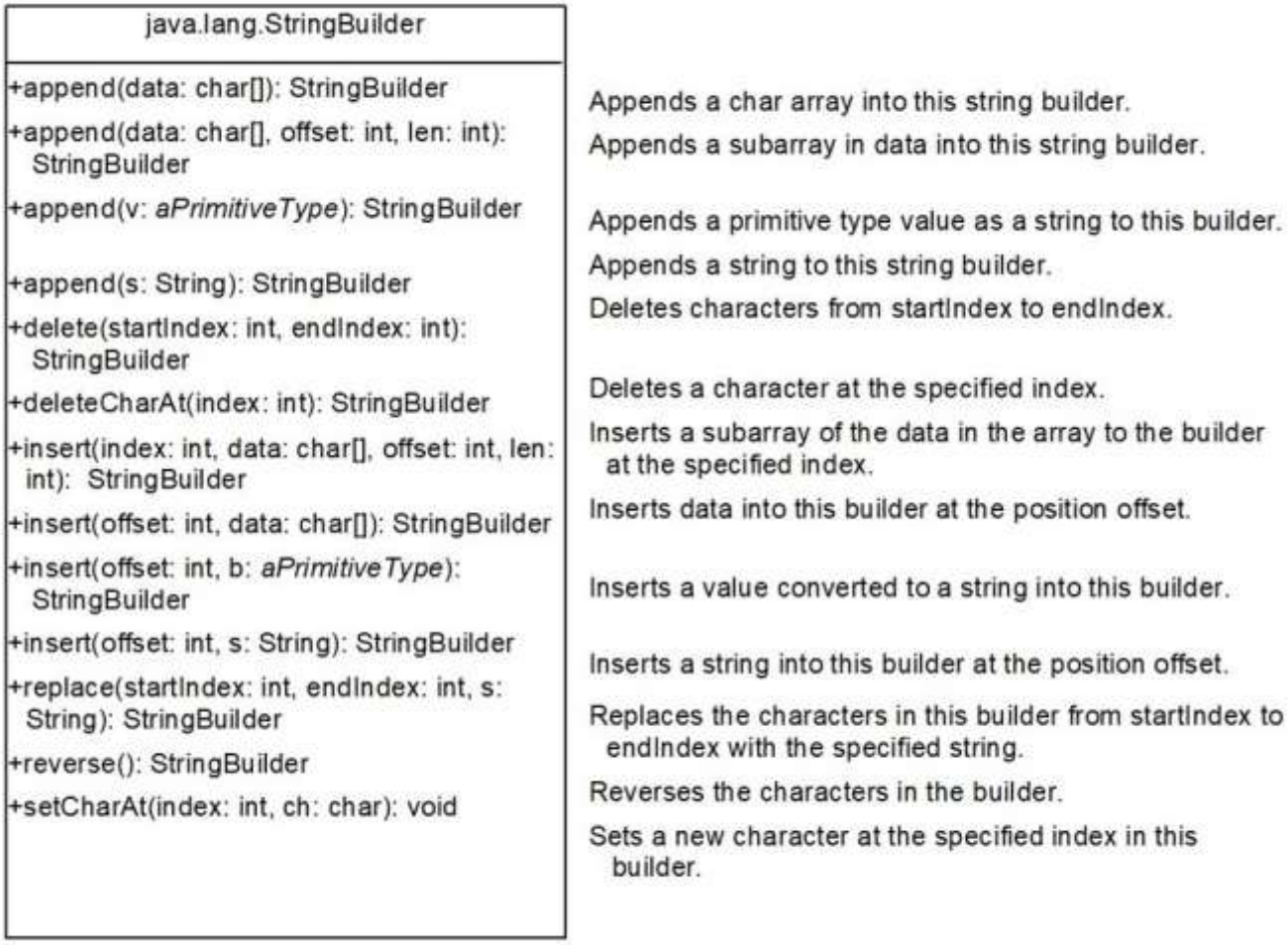
`StringBuilder` Constructors

java.lang.StringBuilder	
+StringBuilder()	Constructs an empty string builder with capacity 16.
+StringBuilder(capacity: int)	Constructs a string builder with the specified capacity.
+StringBuilder(s: String)	Constructs a string builder with the specified string.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Modifying Strings in the Builder



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

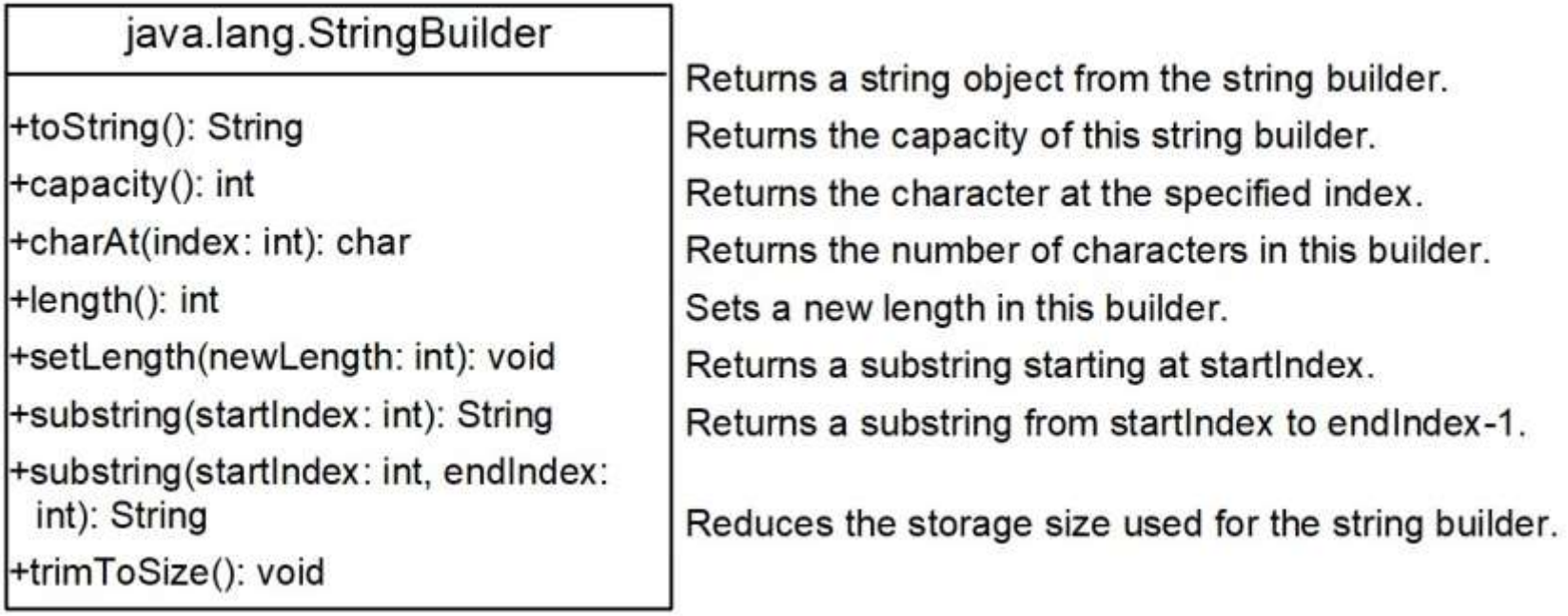
Examples (3 of 4)

```
stringBuilder.append("Java");
stringBuilder.insert(11, "HTML and ");
stringBuilder.delete(8, 11) changes the builder to Welcome
Java.
stringBuilder.deleteCharAt(8) changes the builder to
Welcome o Java.
stringBuilder.reverse() changes the builder to avaJ ot
emocleW.
stringBuilder.replace(11, 15, "HTML")
  changes the builder to Welcome to HTML.
stringBuilder.setCharAt(0, 'w') sets the builder to welcome
to Java.
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `toString`, `capacity`, `length`, `setLength`, and `charAt` Methods



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: Checking Palindromes Ignoring Non-alphanumeric Characters

This example gives a program that counts the number of occurrence of each letter in a string. Assume the letters are not case-sensitive.

PalindromeIgnoreNonAlphanumeric

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Regular Expressions

A **regular expression** (abbreviated **regex**) is a string that describes a pattern for matching a set of strings. Regular expression is a powerful tool for string manipulations. You can use regular expressions for matching, replacing, and splitting strings.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Matching Strings

```
"Java".matches("Java");

"Java".equals("Java");

"Java is fun".matches("Java.*")

"Java is cool".matches("Java.*")

"Java is powerful".matches("Java.*")
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Regular Expression Syntax (1 of 2)

Regular Expression	Matches	Example
x	a specified character **x**	**Java** matches **Java**
.	any single character	**Java** matches **J..a**
(ab|cd)	ab or cd	**ten** matches **t(en|im)**
[abc]	a, b, or c	**Java** matches **Ja[uvwx]a**
[^abc]	any character except a, b, or c	**Java** matches **Ja[^ars]a**
[a-z]	a through z	**Java** matches **[A-M]av[a-d]**
[^a-z]	any character except a through z	**Java** matches **Jav[^b-d]**
[a-e[m-p]]	a through e or m through p	**Java** matches **[A-G[I-M]]av[a-d]**
[a-e&& [c-p]]	intersection of a-e with c-p	**Java** matches **[A-P&& [I-M]]av[a-d]**
\d	a digit, same as [0-9]	**Java2** matches **"Java[\\d]"**
\D	a non-digit	**$Java** matches **"[\\D][\\D]ava"**
\w	a word character	**Java1** matches **"[\\w]ava[\\w]"**
\W	a non-word character	**$Java** matches **"[\\W][\\w]ava"**

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

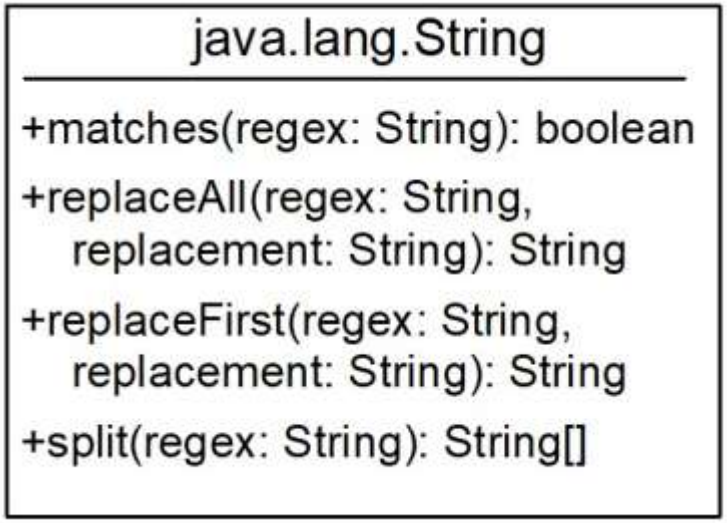
Regular Expression Syntax (2 of 2)

Regular Expression	Matches	Example
`\s`	a whitespace character	**"Java 2"** matches **"Java\\s2"**
`\S`	a non-whitespace char	**Java** matches **"[\\S]ava"**
*p**	zero or more occurrences of pattern *p*	**aaaabb** matches **"a**b"** **ababab** matches **"(ab)*"**
p+	one or more occurrences of pattern *p*	**a** matches **"a+b*"** **able** matches **"(ab)+.*"**
p?	zero or one occurrence of pattern *p*	**Java** matches **"J?Java"** **Java** matches **"J?ava"**
p{n}	exactly n occurrences of pattern *p*	**Java** matches **"Ja{1}.*"** **Java** does not match **".{2}"**
p{n,}	at least n occurrences of pattern *p*	**aaaa** matches **"a{1,}"** **a** does not match **"a{2,}"**
p{n,m}	between n and m occurrences (inclusive)	**aaaa** matches **"a{1,9}"** **abb** does not match **"a{2,9}bb"**

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Replacing and Splitting Strings (2 of 2)



Returns true if this string matches the pattern.

Returns a new string that replaces all matching substrings with the replacement.

Returns a new string that replaces the first matching substring with the replacement.

Returns an array of strings consisting of the substrings split by the matches.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Examples (4 of 4)

```
String s = "Java Java Java".replaceAll("v\\w", "wi") ;

String s = "Java Java Java".replaceFirst("v\\w", "wi") ;

String[] s = "Java1HTML2Perl".split("\\d");
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved